Chapter 29 – Messaging Service

Java Messaging Service (JMS)

The Java Message Service (JMS) defines the standard for reliable Enterprise Messaging.

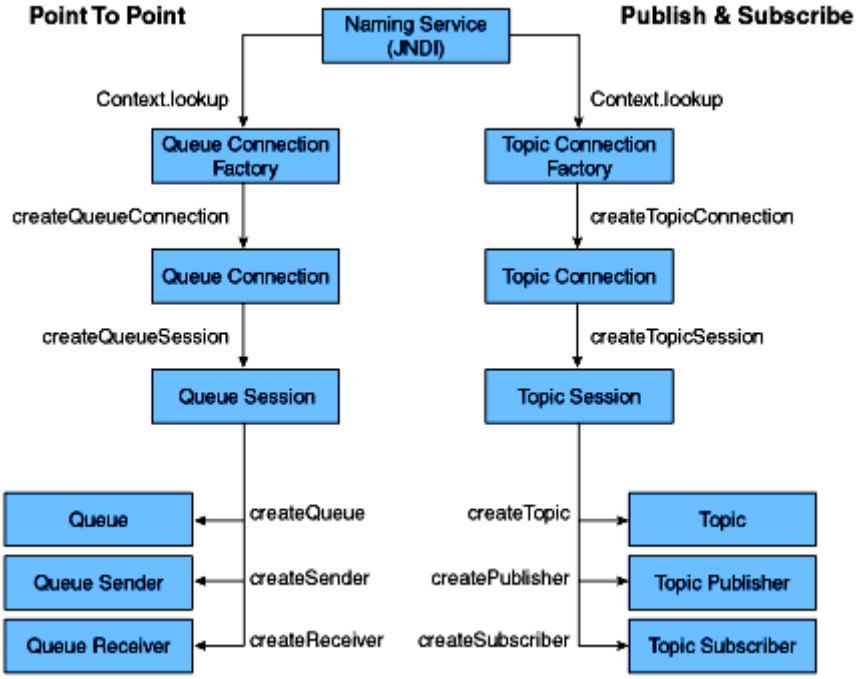
The JMS Elements are:

1. JMS Provider
2. JMS Client
3. JMS Producer/Publisher
4. JMS Consumer/Subscriber
5. JMS Message
6. JMS Queue
7. JMS Topic

The Messaging Models are:

1. Point-to-Point (P2P) or Queuing (Push - Online - Persistence possible)
2. Publish and Subscribe or Topic (Pull - Offline - Store and Forward)

JMS class hierarchy



Chapter 30 – Enterprise Bean

Enterprise Java Beans (EJB)

EJB is a managed, server-side component architecture for modular construction of enterprise applications. It's a server-side model that encapsulates the business logic of an application.

The application server provides:

1. Transaction Processing (XA-1)
2. Integrated with Persistence services (JPA)
3. Concurrent Control
4. Events using JMS
5. Naming and Directory Service (JNDI)
6. Security (JCE & JAAS)
7. RPCs using RMI-IIOP
8. Exposing Business methods as Web Services.

EJB Types

Session Beans
Stateful, Stateless or Singleton accessed via either a Local or Remote or directly without an interface. Support asynchronous execution for all views. It represents a single unit of work or operation.

Message Driven Beans
Message Beans support asynchronous execution, but via a messaging paradigm.

Legacy (Deprecated) Beans
Entity beans were distributed objects having persistent state. **CMP and BMP are two types of Entity beans** that were replaced by the Java Persistence API in EJB 3.0. It represents a single record in the database table wrapped with the business logic.

EJB History

EJB 3.1

- Local view without interface
- .war EJB packaging
- Singleton Session Beans
- Application & Shutdown events
- EJB Timer Service Enhancements
- Asynchronous for session beans

EJB 3.0

- EJBs using annotations rather than complex deployment descriptors
- Use of home, remote interfaces and EJB-jar.XML no longer required.
- Replaced with business interface and a bean that implements it.

EJB 2.1

- Web service support - Stateless session bean invoked over SOAP/HTTP
- EJB Timer Service
- MDBs accepts message from sources other than JMS
- EJB-QL additions - ORDER BY, AVG, SUM, MIN, MAX, COUNT, and MOD
- XML schema replaces DTD as deployment descriptor

EJB 2.0

- Be compatible with CORBA protocols (RMI-IIOP)

EJB 1.1

- XML deployment descriptors
- Default JNDI contexts
- RMI over IIOP
- Security - role driven, not method driven
- Entity Bean Support – mandatory

EJB 1.0 1998

- Defined responsibilities of EJB container, roles, client view and developer's view

Transactions

Six Transaction Attributes

1. Required
2. Requires New
3. Supports
4. Not Supported
5. Mandatory - Transaction Required Exception
6. Never - Remote or EJB Exception

ACID Properties of a Transaction

- Atomicity - Do all or Nothing
- Consistency - Stable Data before/after/failed transactions (Data Integrity)
- Isolation - Isolate current transaction from others
- Durability - Persist the data

Distributed Transactions (XA) & Two-phase Commit Protocol

1. Commit Request Phase (Voting)
2. Commit Phase

Three Different Read Problems

Dirty Reads
One transaction reads rolled back data (stale record) of another transaction.

Non Repeatable Reads
One transaction reads two different values of the same record that was updated by another transaction between two successive reads.

Phantom Reads
One transaction finds different record count between two successive executions due to the record inserted by another transaction during execution.

Transaction Isolation Levels

1. TRANSACTION_READ_UNCOMMITTED – has **all three read problems**
2. TRANSACTION_READ_COMMITTED - prevents **Dirty Reads**
3. TRANSACTION_REPEATABLE_READ - prevents **Dirty Reads and Non Repeatable Reads**
4. TRANSACTION_SERIALIZABLE - prevents **Dirty Reads, Non Repeatable Reads and Phantom Reads**

Performance decreases as the level increases.

Chapter 31 – Persistence

Java Persistence API (JPA)

- javax.persistence package
- JPQL (Java Persistence Query Language)
- Object/Relational metadata

JPA Annotations for Classes
JPA defines three types of persistable classes which are set by the following annotations:

@javax.persistence.Embeddable
@javax.persistence.Entity
@javax.persistence.MappedSuperclass

Entity and mapped super classes can be further configured by annotations that specify cache preferences and lifecycle event listener policy (as explained in chapter 3):

@javax.persistence.Cacheable
@javax.persistence.EntityListeners
@javax.persistence.ExcludeDefaultListeners
@javax.persistence.ExcludeSuperclassListeners

Another JPA class annotation defines an ID class:

@javax.persistence.IdClass

ID classes are useful in representing composite primary keys as explained in the Primary Key section of the ObjectDB manual.

JPA Annotations for JPQL Queries

The following annotations are used to define static named JPA queries:

@javax.persistence.NamedQueries

@javax.persistence.NamedQuery
@javax.persistence.QueryHint

The JPA Named Queries section of the ObjectDB Manual explains and demonstrates how to use these annotations to define named JPQL queries.

JPA Annotations for Fields

The way a field of a persistable class is managed by JPA can be set by the following annotations:

@javax.persistence.Basic
@javax.persistence.Embedded
@javax.persistence.ElementCollection
@javax.persistence.Id
@javax.persistence.EmbeddedId
@javax.persistence.Version
@javax.persistence.Transient

Additional annotations (and enum) are designated for enum fields:

@javax.persistence.Enumerated
@javax.persistence.MapKeyEnumerated
@javax.persistence.EnumType

Other additional annotations (and enum) are designated for date and calendar fields:

@javax.persistence.Temporal
@javax.persistence.TemporalType
@javax.persistence.MapKeyTemporal

JPA Annotations for Relationships

Relationships are persistent fields in persistable classes that reference other entity objects. The four relationship modes are represented by the following annotations:

@javax.persistence.ManyToMany
@javax.persistence.ManyToOne
@javax.persistence.OneToMany
@javax.persistence.OneToOne

Unlike ORM JPA implementations, ObjectDB does not enforce specifying any of the annotations above. Specifying a relationship annotation enables configuring cascade and fetch policy, using the following enum types:

@javax.persistence.CascadeType
@javax.persistence.FetchType

Additional annotations are supported by ObjectDB for the inverse side of a bidirectional relationship (which is calculated by a query):

@javax.persistence.OrderBy
@javax.persistence.MapKey

JPA Annotations for Access Modes

Persistence fields can either be accessed by JPA directly (as fields) or indirectly (as properties and get/set methods). JPA 2 provides an annotation and an enum for setting the access mode:

@javax.persistence.Access
@javax.persistence.AccessType

JPA Annotations for Value Generation

Automatically generated values are mainly useful for primary key fields, but are supported by ObjectDB also for regular (non primary key) persistent fields.

At the field level, the @GeneratedValue with an optional GenerationType strategy is specified:

@javax.persistence.GeneratedValue
@javax.persistence.GenerationType

The @GeneratedValue annotation can also reference a value generator, which is defined at the class level by using one of the following annotations:

@javax.persistence.SequenceGenerator
@javax.persistence.TableGenerator

JPA Annotations for Callback Methods

The following annotations can mark methods as JPA callback methods:

@javax.persistence.PrePersist
@javax.persistence.PreRemove
@javax.persistence.PreUpdate
@javax.persistence.PostLoad
@javax.persistence.PostPersist
@javax.persistence.PostRemove
@javax.persistence.PostUpdate

The Lifecycle Events section of the ObjectDB Manual explains how to use all these annotations on callback methods and with listener classes.

JPA Annotations for Java EE

The following JPA annotations are in use to integrate JPA into a Java EE application and are managed by the Java EE container:

@javax.persistence.PersistenceContext
@javax.persistence.PersistenceContextType
@javax.persistence.PersistenceContexts
@javax.persistence.PersistenceProperty
@javax.persistence.PersistenceUnit
@javax.persistence.PersistenceUnits

Java API for Database Connectivity (JDBC)

Types of JDBC Drivers:

1. JDBC-ODBC Bridge Driver (Bridge) – Type 1
2. Native-API/Partly Java Driver (Native) – Type 2
3. Pure (All) Java, Net-Protocol Driver (Middleware – w3:) – Type 3
4. Pure (All) Java, Native-Protocol Driver (Thin – mysql:) – Type 4

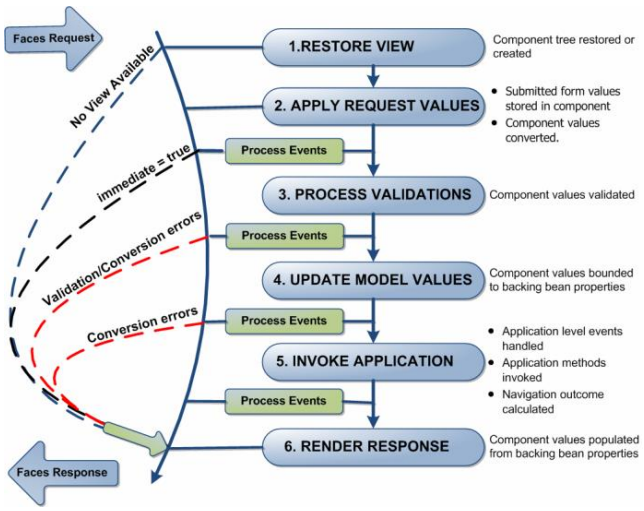
Chapter 32 – User Interface

Java Server Faces (JSF)

A server side user interface component framework for Java™ technology-based web applications. Java Server Faces (JSF) is an industry standard and a framework for building component-based user interfaces for web applications.

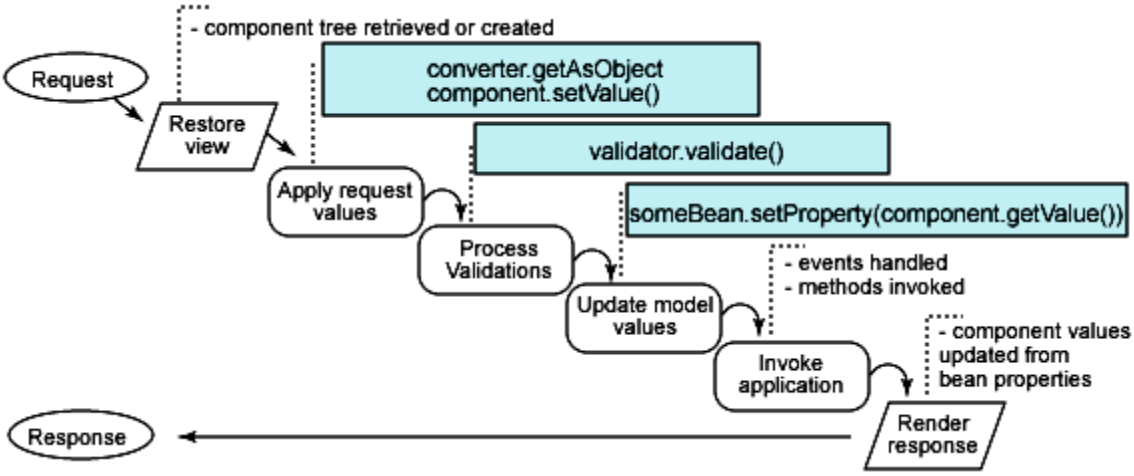
JSF contains an API for representing UI components and managing their state; handling events, server-side validation, and data conversion; defining page navigation; supporting internationalization and accessibility; and providing extensibility for all these features.

JSF Life Cycle



The phases of the JSF application lifecycle are as follows:

1. Restore view
2. Apply request values; process events
3. Process validations; process events
4. Update model values; process events
5. Invoke application; process events
6. Render response



The major benefits of Java Server Faces technology are:

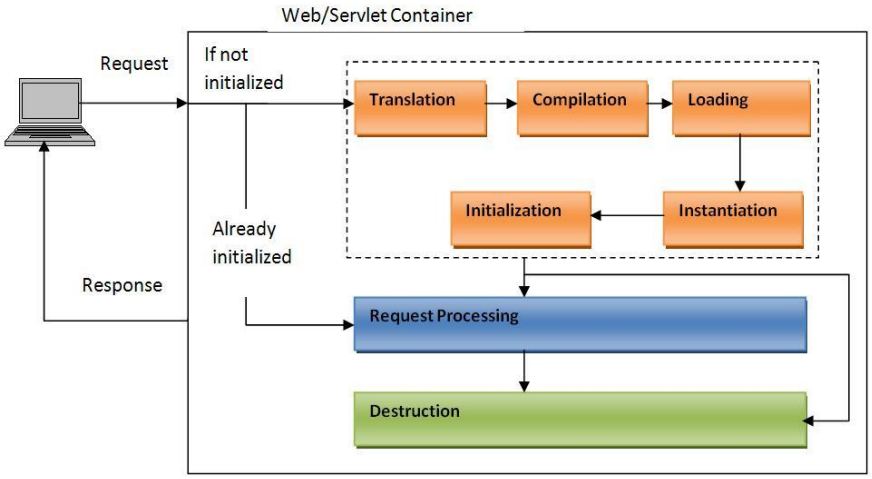
- Java Server Faces architecture makes it easy for the developers to use. In Java Server Faces technology, user interfaces can be created easily with its built–in UI component library, which handles most of the complexities of user interface management.
- Java Server Faces technology offers a clean separation between behavior and presentation.
- Java Server Faces technology provides a rich architecture for managing component state, processing component data, validating user input, and handling events.
- Robust event handling mechanism.
- Events easily tied to server-side code.
- Component-level control over stateful-ness
- Render kit support for different clients
- JSF also supports internationalization and accessibility
- Highly 'pluggable' – components, view handler, etc.
- Offers multiple, standardized vendor implementations

Java Server Pages (JSP)

A server-side technology, Java Server Pages are an extension to the Java servlet technology that was developed by Sun to develop interactive web applications based on HTML, XML, or other document types.

JSP Life Cycle

1. Page translation
2. JSP page compilation
3. Load class
4. Create instance
5. Call jspInit
6. Call _jspService
7. Call jspDestroy



Basically, there are two phases, translation and execution. During the translation phase the container locates or creates the JSP page implementation class that corresponds to a given JSP page. This process is determined by the semantics of the JSP page. The container interprets the standard directives and actions, and the custom actions referencing tag libraries used in the page. A tag library may optionally provide a validation method to validate that a JSP page is correctly using the library. A JSP container has flexibility in the details of the JSP page implementation class that can be used to address quality-of-service and most notably, the performance issues.

During the execution phase the JSP container delivers events to the JSP page implementation object. The container is responsible for instantiating request and response objects and invoking the appropriate JSP page implementation object. Upon completion of processing, the response object is received by the container for communication to the client.

JSP Tag Types

1) Directive Tags
These tags control translation and compilation phases.
<%@Directive {attribute="value"}* %>
<%@page …%> <jsp:directive.page …/>
<%@taglib uri=http://my.com/my.tld prefix="my" %>
<%@include page="enterRecord.jsp" flush=true"%>

2) Declaration Tags
These tags declare an instance java variable or method. No output produced
<%! JavaDeclaration %> equivalent to XML tag <jsp:declaration> … </jsp:declaration>

3) Scriptlets
These tags have valid java statements. No output produced unless out is used.
<% JavaStatements %> equivalent to XML tag <jsp:scriptlet> … </jsp:scriptlet>

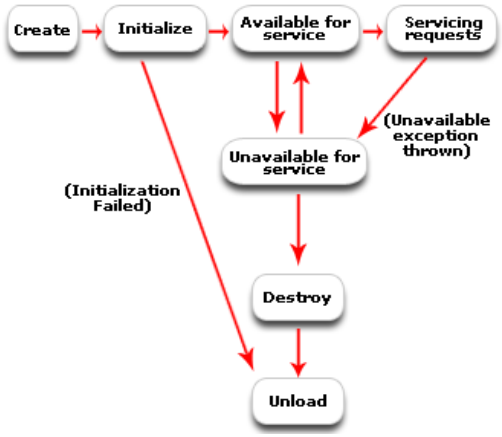
4) Expressions
These tags evaluate a Java expression to a String and then included in the output.
<%= JavaExpression %> equivalent to XML tag <jsp:expression> … </jsp:expression>

5) Action Tags (Bean <jsp:useBean>, Include <jsp:include> Forward <jsp:forward>) and

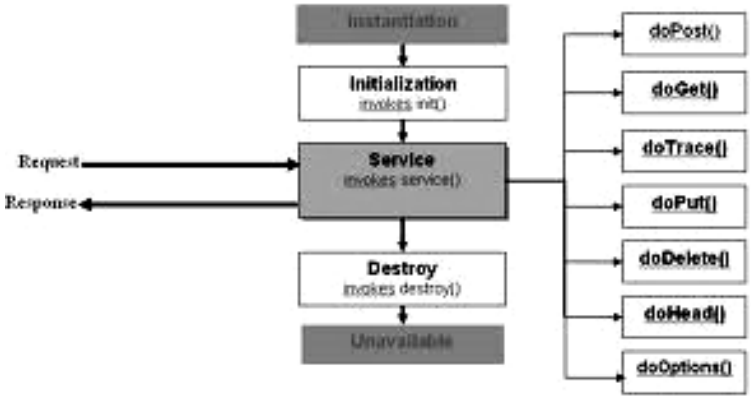
6) Comments <%-- --%>

Implicit Objects Variables

Implicit Object	Type	Purpose/Useful methods	Scope
request	Subclass of ServletRequest	The HTTP request sent to the server from the client	request
response	Subclass of ServletResponse	The HTTP response sent to the client from the server	page
out	JspWriter	Object for writing to the output stream. flush, getBufferSize	page
session	HttpSession	created as long as <% page session="false" %> is *not* used. Valid for Http protocol only. getAttribute, setAttribute	session
config	ServletConfig	Specific to a servlet instance. Same as Servlet.getServletConfig() getInitParameter, getInitParameterNames	page
application	ServletContext	Available to all Servlets (application wide). Provides access to resources of the servlet engine (resources, attributes, context params, request dispatcher, server info, URL & MIME resources).	application
page	Object	this instance of the page (equivalent servlet instance 'this').	page
pageContext	PageContext	provides a context to store references to objects used by the page, encapsulates implementation-dependent features, and provides convenience methods.	page
exception	java.lang.Throwable	created only if <%@ page isErrorPage="true" %>	page



Servlet Life Cycle



Chapter 33 – Security

J2EE Security Model

The J2EE platform architecture provides for the secure deployment of application components. It emphasizes the declarative approach wherein the application components' security structure, roles, access control, authentication and authorization requirements - as well as the other characteristics pertaining to transactions, persistence, and more - are expressed and managed outside the application code.

J2EE server products provide deployment tools that support the declarative customization of application components for the operational runtime environment. An application component provider isn't expected to implement security services, as it's the responsibility of the J2EE container and server. The security functions provided by the J2EE platform include authentication, access authorization, and secure communication with clients.

Authentication

Authentication is the process by which an entity (such as a user, organization, or program) proves and establishes its identity with the system by supplying authentication data. For users this typically means a user name and password. (In general, authentication data could be comprised of a digital certificate, or even biometric data such as a fingerprint, iris, or retina scan.) A principal is an entity that can be validated by an authentication mechanism; it's identified using a principal name and verified using authentication data.

Types of Authentication:

1. Basic
2. Form-Based
3. Digest
4. SSL and Client Certification

Web Container's Support for Authentication

The <auth-method> element in the Web deployment descriptor is used to configure the type of authentication mechanism such as "BASIC", and "FORM".

<auth-method>FORM</auth-method>

The deployment tools provided with the J2EE server product insulate us from having to manually write the deployment descriptors; the elements are provided here for reference. The following are some of the authentication mechanisms made available by the Web container and server.

Basic Authentication

HTTP basic authentication is the simplest. When a user attempts to access any protected Web resource, the Web container checks if the user has already been authenticated. If the user hasn't, the browser's built-in login screen is used to solicit the user name and password from the user so that the Web server can perform authentication. If the login fails, the browser's built-in screens and messages will be used. The user name and password are sent using simple base 64 encoding.

Form-Based Authentication

Form-based authentication is used if an application-specific login screen is required, and the browser's built-in authentication screen isn't adequate. The Web deployment descriptor needs to specify the login form page and the error page to be used with this mechanism. When the user attempts to access any protected Web resources, the Web container checks if the user has already been authenticated. If the user hasn't, the container presents the login form as specified in the deployment descriptor. If authentication fails, the error page, as specified in the deployment descriptor, is displayed.

The <form-login-page> and <form-error-page> elements are respectively used to specify the location of the login and error pages that need to be displayed. Further, the login page must contain fields named precisely j_username and j_password to represent the user name and password, respectively, as shown in Listing 1.

HTTPS Authentication

HTTPS (HTTP over SSL) authentication is a strong authentication mechanism. It requires the user to possess a public key certificate and is ideal for e-commerce applications as well as single sign-ons from within the browser in an enterprise.

Hybrid Authentication

In both the HTTP basic and the form-based authentication, passwords aren't adequately protected for confidentiality. This deficiency can be overcome by running HTTP basic and HTTP form-based authentication mechanisms over SSL. Generally, the use of the CONFIDENTIAL flag in the <transport-guarantee> element of the Web deployment descriptor ensures the use of SSL for data transmission.

Authorization and Access Control

Authorization and access control mechanisms ensure that only authenticated principals who have the required permissions to access application components are able to do so. In general, there are two fundamental approaches to controlling access - capabilities and permissions. The capabilities-based approach focuses on what resources a given user can access. The permissions-based approach on the other hand focuses on which users can access a given resource. The J2EE authorization model uses role-based permissions. A role is a logical grouping of users that's used to define a logical security view for the application. Protected application resources have associated authorization rules - roles that are allowed to access a given component are specified in the application component's deployment descriptor. The deplorer maps the roles to actual users using the J2EE server's deployment tools. The J2EE server enforces

the prescribed security policies at runtime and ensures that only those users who belong to the appropriate role are able to access the protected components' functionality - while those who don't belong are denied access.

J2EE Containers

A container is part of the J2EE server. It provides deployment and runtime support for application components and is responsible for infrastructural services including security. There are three types of J2EE containers that are meant to house different J2EE application components:

EJB container: Houses EJB components
Web container: Houses servlets, JSPs, static HTML, JPEG files, and more
J2EE application client container: Houses J2EE application clients

Each of these containers has its associated deployment descriptor.

Obtaining the Initiating Security Context

In the J2EE model, secure applications require that the client programs be authenticated. An end user can be authenticated using either a Web or an application client. Once the user is authenticated, an initial security context is generated and maintained by the J2EE platform. This security context is the encapsulation of the identity of the authenticated user principal.

Chapter 34 – SOAP and REST

Simple Object Access Protocol (SOAP)

SOAP is a simple XML-based protocol designed to let applications exchange information over HTTP. It is a protocol for accessing the Web Services.

RPC (DCOM or CORBA) is used to communicate between distributed applications. But, it possesses compatibility and security problems. It gets blocked by firewalls and proxy servers. Better way is HTTP because it is supported by all browsers and servers. But, it wasn't designed for this purpose. So, SOAP was created to accomplish this.

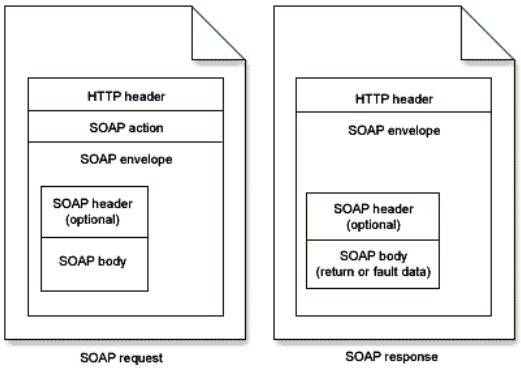
SOAP provides a way to communicate between **applications** running on **different operating systems**, with **different technologies** and **programming languages**.

A SOAP message consists of the following four parts:

ENVELOPE - it is a mandatory part defining the beginning and end of the message.
HEADER - It is optional and if included, may contain any optional properties (attributes) that were included in the building of the message and which may later be required in processing the message.
BODY - It is a mandatory part and contains the XML containing the message to be sent.
FAULT - Again, this is an optional component, providing additional information about errors that occurred whilst processing the message.

SOAP Data Types

1. Scalar types: Contains exactly one value, e.g., a last name, price.
2. Compound types: Contain multiple values, such as a purchase order



For message transmission, SOAP makes use of an 'internet application layer' protocol as a 'message transfer protocol'. Both SMTP and HTTP are widely used as these application layer protocols with HTTP being the more preferred one, as it works better with network firewalls.

```
<soap:Envelope xmlns:soap soap:encodingStyle>
   <soap:Header>
     <m:Trans xmlns:m soap:mustUnderstand=0|1/>
    </soap:Header>
   <soap:Body>
       <soap:Fault>
          <faultcode>
          <faultstring>
          <faultactor>
          <detail>
       </soap:Fault>
   <soap:Body>
</soap:Envelope>
```

SOAP Building Blocks

- an Envelope element that identifies the XML document as a SOAP message
- a Header element that contains header information (authentication,payment, and so on)
- a Body element that contains call and response information
- a Fault element containing error and status information

Syntax Rules of a SOAP Message

1. Must be encoded using XML
2. Must use SOAP Envelope namespace
3. Must use SOAP Encoding namespace
4. Must not contain a DTD reference
5. Must not contain XML Processing Instructions

SOAP Fault Codes

1. VersionMismatch
2. MustUnderstand
3. Client
4. Server

SOAP HTTP Binding

HTTP + XML = SOAP
Content-Type: application/soap+XML

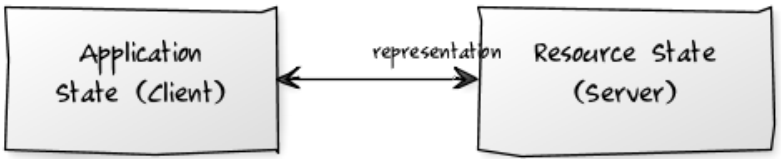
Representational State Transfer (REST)

RESTFul web services are based on the HTTP methods and concept of REST. REST follows one-to-one mapping between CRUD (Create, Read, Update & Delete) operations and HTTP methods.

- To create a resource on the server, use POST
- To retrieve a resource, use GET
- To change the state of a resource or to update it, use PUT
- To remove or delete a resource, use DELETE

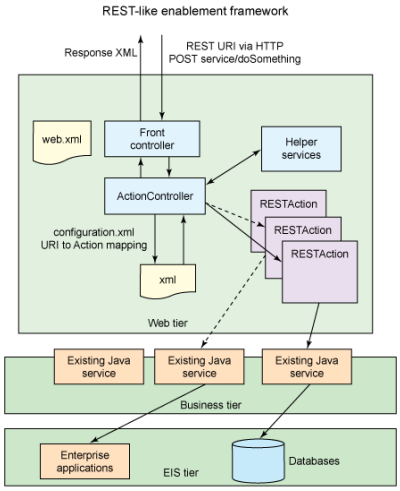
REpresentational State Transfer (REST) is an architecture principle in which the web services are viewed as resources and can be uniquely identified by their URLs.

Representational State Transfer refers to transferring "representations". You are using a "representation" of a resource to transfer resource state which lives on the server into application state on the client.



The most important concept in REST is resources, which are identified by unique global IDs— typically using URIs. Client applications use HTTP methods (GET/ POST/ PUT/ DELETE) to manipulate the resource or collection of resources. A RESTful Web service is a web service implemented using HTTP and the principles of REST. Typically, a RESTful Web service should define the following aspects:

- The base/root URI for the Web service such as http://host/<appcontext>/resources.
- The MIME type of the response data supported, which are JSON/XML/ATOM and so on.
- The set of operations supported by the service. (for example, POST, GET, PUT or DELETE).



The key characteristic of a RESTful Web service is the explicit use of HTTP methods to denote the invocation of different operations.

The basic REST design principle uses the HTTP protocol methods for typical CRUD operations:

- POST - Create a resource
- GET - Retrieve a resource
- PUT – Update a resource
- DELETE - Delete a resource

The major advantages of REST-services are:

- They are **highly reusable across platforms** (Java, .NET, PHP, etc) since they rely on basic **HTTP** They use **basic XML** instead of the complex SOAP XML and are easily consumable

REST-based web services are increasingly being preferred for integration with backend enterprise services. In comparison to SOAP based web services, the programming model is simpler and the use of native XML instead of SOAP reduces the serialization and deserialization complexity as well as the need for additional third-party libraries for the same.

Features of REST:

- Light-weight
- No WSDL file
- Only XML file and no SOAP envelope or header that reduces the processing overhead
- Supports only HTTP and not FTP or SMTP
- Supports MIME types such as text/XML, text/JSON, and so on
- If enabled, then both REST and SOAP endpoints are enabled

Chapter 35 – JAXB and JAXP

Java Architecture for XML Binding (JAXB)

Java Architecture for XML Binding (JAXB) is a Java standard that defines how Java objects are converted from and to XML. It uses a standard set of mappings. JAXB defines an API for reading and writing Java objects to and from XML documents.

Key Annotations

Annotation	Description
@XmlSchema	Maps a package name to a XML namespace.
@XmlRootElement (namespace = "namespace")	Defines the root element for an XML tree. It maps a class or an enum type to an XML element.
@XmlAccessorOrder	Controls the ordering of fields and properties in a class.
@XmlType	Maps a class or an enum type to a XML Schema type.
@XmlType(propOrder = { "field2", "field1",.. })	Allows defining the order in which the fields are written in the XML file.
@XmlElement(name = "neuName")	Maps a JavaBean property to a XML element derived from property name.
@XmlElementWrapper(name = "bookList")	Generates a wrapper element around XML representation.
@XmlElements	A container for multiple @XmlElement annotations.
@XmlAttribute	Maps a JavaBean property to a XML attribute.
@XmlAttachmentRef	Marks a field/property that its XML form is an URI reference to mime content.
@XmlMimeType	Associates the MIME type that controls the XML representation of the property.
@XmlEnum	Maps an enum type Enum to XML representation.
@XmlEnumValue	Maps an enum constant in Enum type to XML representation.

Code Snippet that maps the Java Bean to XML document

```
@XmlRootElement(name = "book")
// If you want you can define the order in which the fields are written (Optional)
@XmlType(propOrder = { "author", "name", "publisher", "isbn" })
public class BookStore {

 private String storeName;

 // XmLElementWrapper generates a wrapper element around XML representation
 @XmlElementWrapper(name = "bookList")
 // XmlElement sets the name of the entities
 @XmlElement(name = "book")
 private ArrayList<Book> bookList;

 // If you like the variable name, e.g. "name", you can easily change this name for your XML-Output:
 @XmlElement(name = "title")
 public String getStoreName() {
  return storeName;
 }

}
```

Code Snippet that converts the Java Object from and to XML document

```
private static final String BOOKSTORE_XML = "./bookstore-jaxb.xml";

 try{

  // Create JAXB context and load the book store object
  JAXBContext context = JAXBContext.newInstance(Bookstore.class);

  // Instantiate Marshaller and Write to XML file
  Marshaller m = context.createMarshaller();
  m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, Boolean.TRUE);
  m.marshal(bookstore1, new File(BOOKSTORE_XML));

  // Instantiate Unmarshaller and Read from XML file
  Unmarshaller um = context.createUnmarshaller();
  Bookstore bookstore2 = (Bookstore) um.unmarshal(new FileReader(BOOKSTORE_XML));

} catch (JAXBException jaxbe, IOException ioe) {}
```

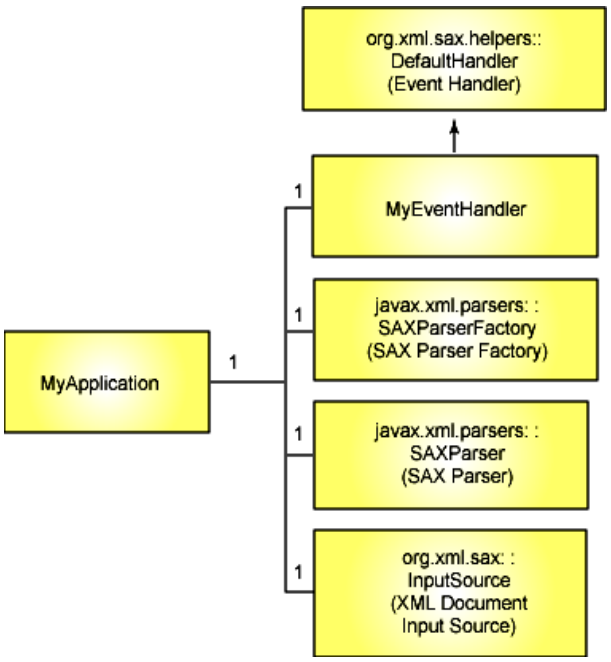
Types of XML Parsers

1. SAX (Simple API for XML)
2. DOM (Document Object Model)

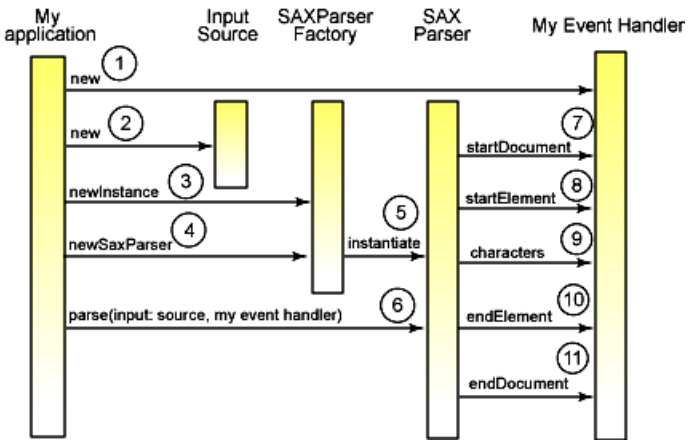
Simple API for XML (SAX)

1. Event based model
2. Serial access (flow of events)
3. Low memory usage
4. To process parts of the document
5. To process the document only once.

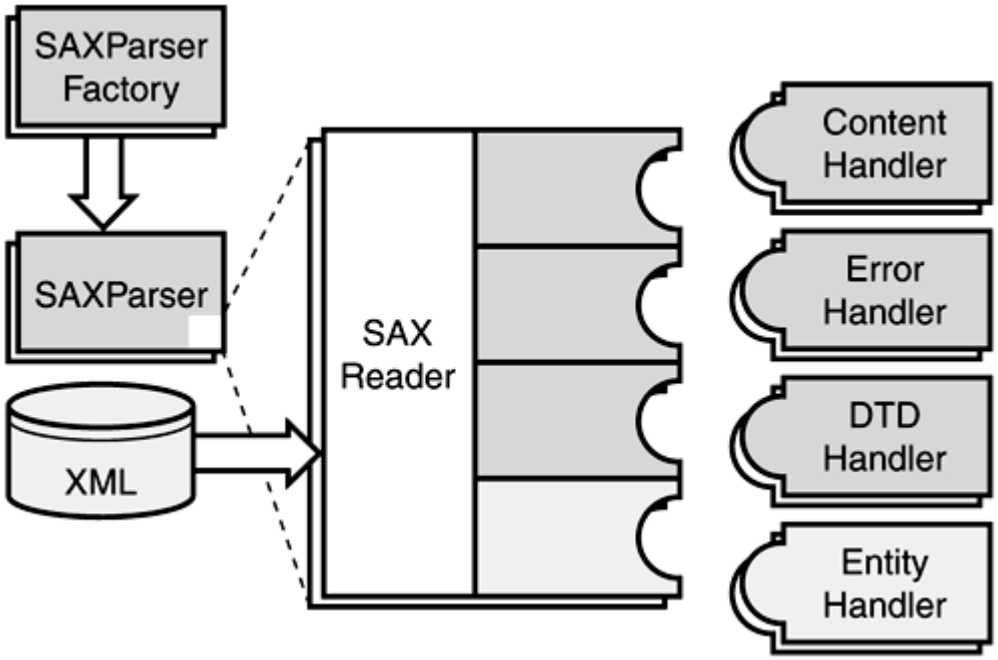
Typical elements of a JAXP application



JAXP Sequence Diagram



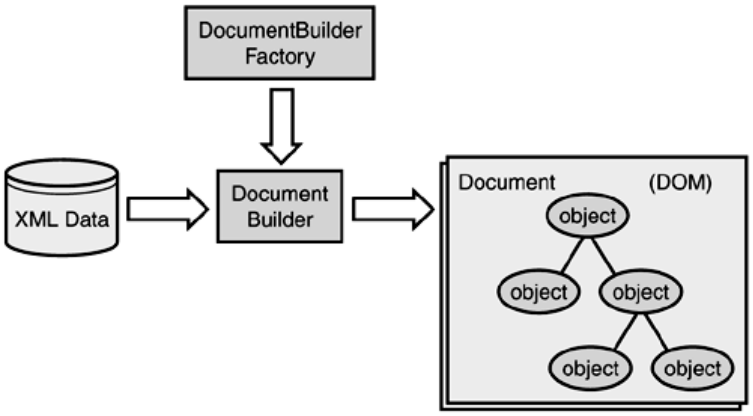
SAX parsing architecture



Document Object Model (DOM)

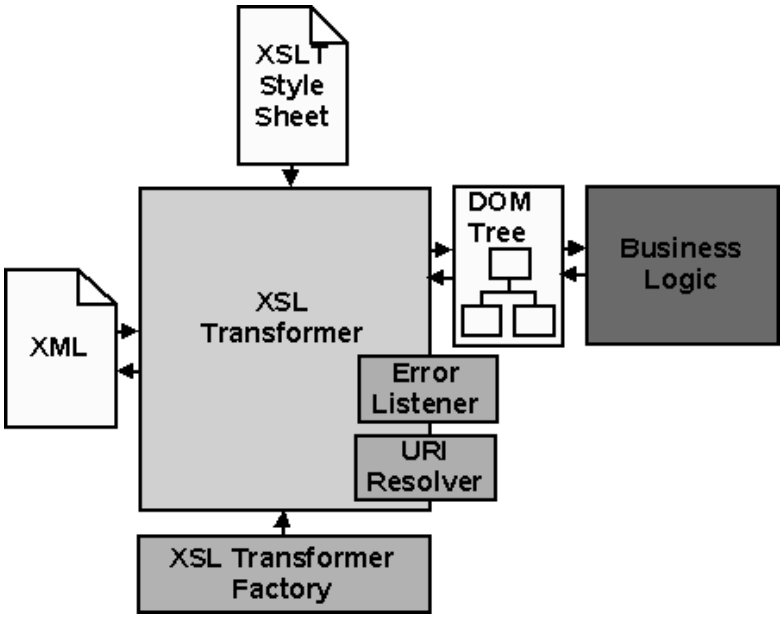
1. Tree data structure
2. Random access
3. High memory usage
4. To edit the document
5. To process the document multiple times

DOM parsing architecture



XSLT - XML (Extensible Markup Language) Stylesheet Language for Transformations

1. Based on SAX or DOM
2. Xpath patterns
3. XSL statements



Chapter 36 – Web Service

A web service is a method of communication between two electronic devices over the web (internet).

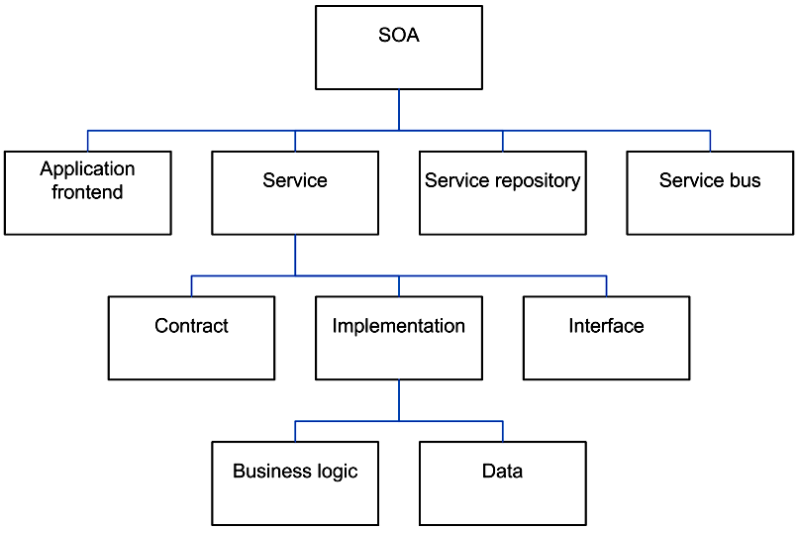
Web services are a set of tools that can be used in a number of ways. There are three most common styles of use. They are:

- Remote procedure calls (RPC) – Java API for XML based RPC (JAX-RPC)
- Service-Oriented Architecture (SOA) – Java API for XML based Web Services (JAX-WS)
- Representational State Transfer (REST) – Java API for Restful Web Services (JAX-RS)

Service Oriented Architecture (SOA)

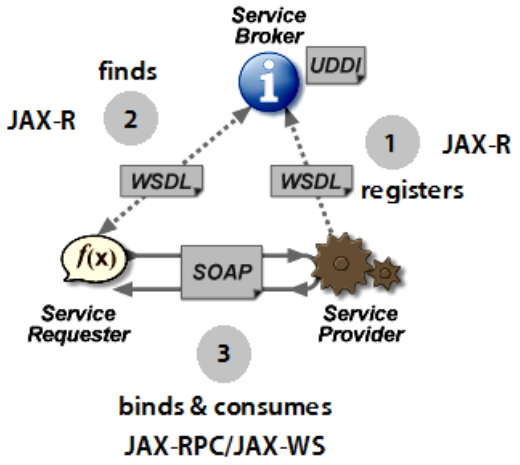
In software engineering, a Service-Oriented Architecture (SOA) is a set of principles and methodologies for designing and developing software in the form of interoperable services.

Elements of Service-Oriented Architecture



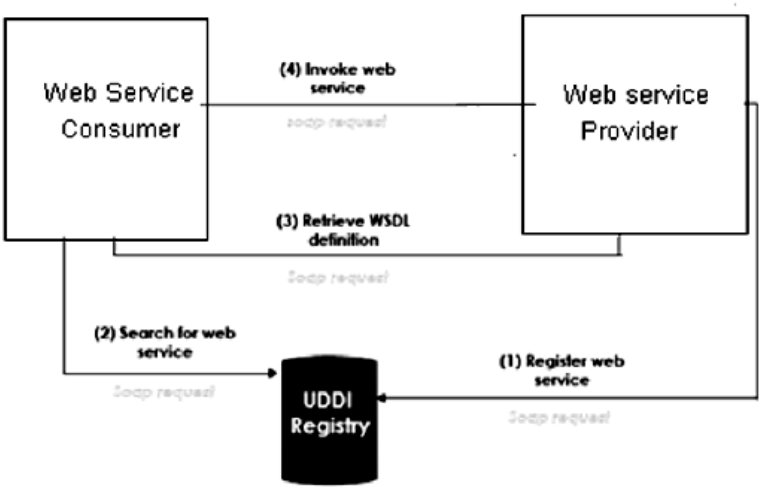
Three critical roles in SOA:

1. Service Provider
2. Service Broker
3. Service Consusmer or Requester



General Flow:

1. Service Provider **registers** the service contract (WSDL) with Service Broker (UDDI registry) using JAX-R
2. Service Consumer **finds** the service contract (WSDL) registered with Service Broker (UDDI registry) using JAX-R
3. Service Consumer or Requester (Client) **binds** and **consumes** the service provided by Service Provider using SOAP/SAAJ protocol and JAX-RPC/JAX-WS APIs



Glossary

Term	Description
WSDD	Web Service Deployment Descriptor (services.xml)
SAAJ	SOAP with Attachments API for Java
UDDI	Universal Description, Discovery and Integration
WSDL	Web Service Description Language (for SOAP based web services)
SOAP	Simple Object Access Protocol
REST	Representational State Transfer
JAX-RPC	Java API for XML-based Remote Procedure Call
JAX-WS	Java API for XML-based Web Services
JAX-RS	Java API for RESTful web Services
JAX-R	Java API for XML-based Registry
JAXB	Java Architecture for XML Binding
WS-S	Web Services - Security
MTOM	Message Transmission Optimization Mechanism
WS-I BP	Web Services-Interoperability (Basic Profile)
WADL	Web Application Description Language (for RESTful web services)

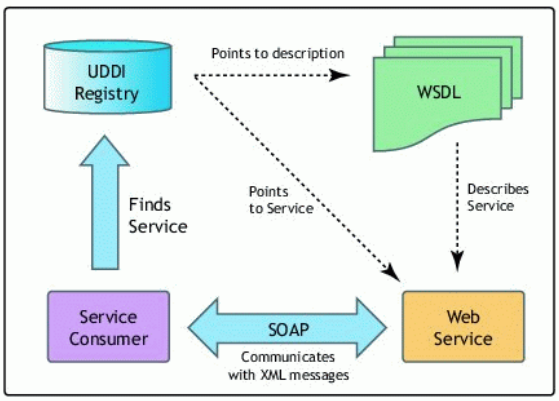
Web Service Deployment Descriptor (WSDD)

WSDD (services.xml) contains

- service provider information (ex: Java: RPC)
- class name
- allowable method names
- scope

Universal Description, Discovery and Integration (UDDI)

UDDI is a platform-independent, XML-based registry and directory service where businesses can register and search for web services.



Web Service Approaches:

1. Top Down Development (Contract First)
2. Bottom Up Development (Code First)

Top Down Development

Top-down web service development involves creating a web service from a WSDL file. By creating the WSDL file first you will ultimately have more control over the web service.

In top-down approach, the WSDL is **designed from a business point of view** and is not driven by a service consumer or an existing application. The “WSDL to Java” tool generates java code according to JAX-RPC or JAX-WS. These classes can then adapt the existing business logic.

Bottom Up Development

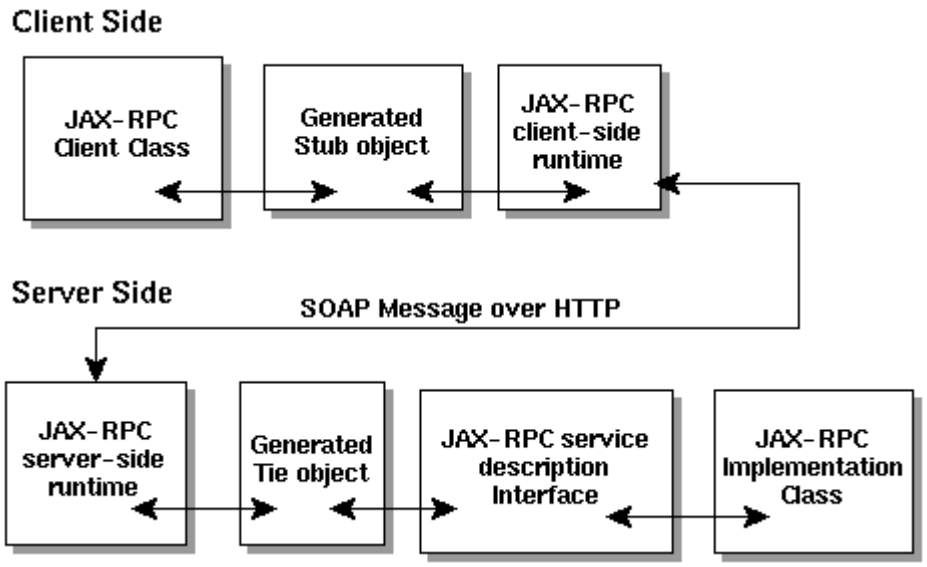
In bottom-up approach, first you create a POJO or EJB and then use the web services wizard to create the WSDL file and Web service.

Although bottom-up web service development may be faster and easier, especially if you are new to Web services, the top-down approach is the recommended way of creating a Web service. The bottom up approach generates web services for which the design is driven from an **application point of view** (developer driven) and will not address the need for other consumers.

Java API for XML based RPC (JAX-RPC)

It allows a Java application to invoke Java-based Web Services with a known description while still being consistent with its WSDL description. It's an API for building Web Services and clients that used RPC and XML.

JAX-RPC Flow Diagram



It works as follows:

1. A Java program invokes a method on a stub
2. The stub invokes routines in the JAX-RPC Client-side Runtime System (RS)
3. The RS converts the remote method invocation into a SOAP message
4. The RS transmits the message as an HTTP request
5. The JAX-RPC Server-side Runtime System (RS) receives the SOAP message and converts the request into server method call
6. The RS invokes the appropriate method on a tie object
7. The tie object invokes business method in the service implementation class through the service description interface

Thus, Servlet, EJB, JB or POJO are made available through Web Services.

Java API for XML based Web Services (JAX-WS)

JAX-RPC 2.0 was renamed JAX-WS 2.0 for Message-oriented Web Services and also supports SOAP 1.1 & 1.2 with WS-I BP 1.1, JAXB2.0 for mapping, MTOM and Java 5.0.

Figure 1: JAX-WS

JAX-RS uses annotations to define the REST relevance of Java classes. The package name is **javax.ws.rs.**

Annotation	Description	Usage

@GET	Indicates that the following method will answer to an HTTP GET/Retrieve request.	**@GET**
		public String getHTML() {
		...
		}

@POST	Indicates that the following method will answer to an HTTP POST/Create request.	**@POST**
		@Consumes("application/json")
		@Produces("application/json")
		public RestResponse<Contact> create(Contact contact) {
		...
		}

@PUT	Indicates that the following method will answer to an HTTP PUT/Update request.	**@PUT**
		@Consumes("application/json")
		@Produces("application/json")
		@Path("{contactId}")
		public RestResponse<Contact> update(Contact contact) {
		...
		}

@DELETE	Indicates that the following method will answer to an HTTP DELETE/Remove request.	**@DELETE**
		@Produces("application/json")
		@Path("{contactId}")
		public RestResponse<Contact>
		delete(@PathParam("contactId") int contactId) {
		...
		}

@Consumes (MediaType.APPLICATION_XML[, more-types])	'Consumes' defines which MIME type is consumed by a method annotated with @POST, @PUT or @DELETE.	@PUT
		@Consumes("application/json")
		@Produces("application/json")
		@Path("{contactId}")
		public void delete(Contact contact) {
		...
		}

@Produces (MediaType.TEXT_PLAIN[, more-types])	'Produces' defines which MIME type is delivered or prodcued by a method annotated with @GET. The standard outputs are "text/plain", "text/html", "application/xml" and "application/json".	@GET
		@Produces("application/json")
		public Contact getJSON() {
		...
		}

@Path(your_path)	Specifies the URL path on which this method will be invoked. The base URL is based on your application name, the servlet and the URL pattern from the web.xml configuration file.	@GET
		@Produces("application/xml")
		@Path("users/{username: [a-zA-Z][a-zA-Z_0-9]*}")
		public Contact getXML() {
		...
		}

@PathParam	Used to inject values from the URL into a method parameter. We can bind REST-style URL parameters to method arguments using @PathParam annotation.	@GET
		@Produces("application/xml")
		@Path("xml/{firstName}")
		public Contact
		getXML(**@PathParam("firstName") String firstName**) {
		Contact contact =
		contactService.findByFirstName(firstName);
		return contact;
		}

@QueryParam	Request parameters in query string can be accessed using @QueryParam annotation.	@GET
		@Produces("application/json")
		@Path("json/companyList")
		public CompanyList
		getJSON(**@QueryParam("start") int start**,
		@QueryParam("limit") int limit) {
		CompanyList list = new
		CompanyList(companyService.listCompanies(start, limit));
		return list;
		}

@FormParam	The REST resources will usually consume XML/JSON for the complete Entity Bean. Sometimes, you may want to read parameters sent in POST requests directly using @FormParam.	@POST
		public String save(**@FormParam("firstName") String firstName**,
		@FormParam("lastName") String lastName) {
		...
		}

@HeaderParam	Binds the value(s) of a HTTP header to a resource method parameter, resource class field, or resource class bean property.	public class MyBeanParam {
		@HeaderParam("header")
		private String headerParam;
		}

@CookieParam	Binds the value of a HTTP cookie to a resource method parameter, resource class field, or resource class bean property.	public class MyBeanParam {
		@CookieParam("cookieName")
		private String cookieParam;
		}

@MatrixParam	Binds the value(s) of a URI matrix parameter to a resource method parameter, resource class field, or resource class bean property.	public class MyBeanParam {
		@MatrixParam("m")
		@Encoded
		@DefaultValue("default")
		private String matrixParam;
		}

@BeanParam	It can contain all parameter injections. The JAX-RS runtime will instantiate the object and inject all its fields and properties annotated with either one of the @XxxParam annotation or the @Context annotation.	public class MyBean {
		@FormParam("myData")
		private String data;
		@HeaderParam("myHeader")
		private String header;
		@PathParam("id")
		public void setResourceId(String id) {...}
		}
		
		@Path("myresources")
		public class MyResources {
		@POST
		@Path("{id}")
		public void post**(@BeanParam MyBean myBean**) {...}
		}

@DefaultValue	Defines the default value of request meta-data that is bound using either one of PathParam, QueryParam, MatrixParam, CookieParam, FormParam or HeaderParam.	@Path("{id:\\d+}")
		public class InjectedResource {
		// Injection onto field
		@DefaultValue("q") @QueryParam("p")
		private String p;
		}

@Encoded	Disables automatic decoding of parameter values bound using QueryParam, PathParam, FormParam or MatrixParam.	public class MyBeanParam {
		@MatrixParam("m")
		@Encoded
		@DefaultValue("default")
		private String matrixParam;
		}

@Context	This annotation is used to inject information into a class field, bean property or method parameter. When deploying a	@GET public String get(**@Context UriInfo ui**) { MultivaluedMap<String, String> queryParams = ui.getQueryParameters();

	JAX-RS application using servlet then ServletConfig, ServletContext, HttpServletRequest and HttpServletResponse are available using @Context.	MultivaluedMap<String, String> pathParams = ui.getPathParameters();
		}
		
		@GET
		public String get**(@Context HttpHeaders hh**) {
		MultivaluedMap<String, String> headerParams = hh.getRequestHeaders();
		Map<String, Cookie> pathParams = hh.getCookies();
		}
		
		@Context
		public void setRequest(**Request request**) {
		String[] requestParams = request.getRequestParams();
		}
		
		public MySingletonResource**(@Context SecurityContext securityContext**) {
		…
		}

@Singleton	In this scope there is only one instance per jax-rs application.	**@Singleton**
		@Path("/printers")
		public class PrintersResource {
		...
		}

@RequestScoped	Default lifecycle (applied when no annotation is present). In this scope the resource instance is created for each new request and used for processing of this request.	**@RequestScoped**
		@Path("/pathUrl")
		public class UserAction {
		...
		}

@PerLookup	In this scope the resource instance is created every time it is needed for the processing even it handles the same request.	**@PerLookup**
		@Path("/pathUrl")
		public class GranularTransaction {
		...
		}

Chapter 37 – Java Script

What is Java Script?

JavaScript is a lightweight programming language and it is designed to add interactivity to HTML pages and usually embedded directly into HTML pages. It is an interpreted language (means that scripts execute without preliminary compilation).

- **JavaScript gives HTML designers a programming tool** - HTML authors are normally not programmers, but JavaScript is a scripting language with a very simple syntax! Almost anyone can put small "snippets" of code into their HTML pages
- **JavaScript can react to events** - A JavaScript can be set to execute when something happens, like when a page has finished loading or when a user clicks on an HTML element
- **JavaScript can read and write HTML elements** - A JavaScript can read and change the content of an HTML element
- **JavaScript can be used to validate data** - A JavaScript can be used to validate form data before it is submitted to a server. This saves the server from extra processing
- **JavaScript can be used to detect the visitor's browser** - A JavaScript can be used to detect the visitor's browser, and - depending on the browser - load another page specifically designed for that browser
- **JavaScript can be used to create cookies** - A JavaScript can be used to store and retrieve information on the visitor's computer

Key Notes

- Unlike HTML, JavaScript is case sensitive - therefore watch your capitalization closely when you write JavaScript statements, create or call variables, objects and functions.

- The HTML <script> tag is used to insert a JavaScript into an HTML page.

- Try to avoid using document.write() in real life JavaScript code. The entire HTML page will be overwritten if document.write() is used inside a function, or after the page is loaded. However, document.write() is an easy way to demonstrate JavaScript output in a tutorial.

- To manipulate HTML elements JavaScript uses the DOM method getElementById(). This method accesses the element with the specified id.
- External script cannot contain the <script></script> tags
- Using semicolons makes it possible to write multiple statements on one line.
- Comments can be added to explain the JavaScript, or to make the code more readable. Single line comments start with //. Multi line comments start with /* and end with */.
- Variable names are case sensitive (y and Y are two different variables)
- Variable names must begin with a letter, the $ character, or the underscore character
- Because JavaScript is case-sensitive, variable names are case-sensitive.
- When you assign a text value to a variable, put quotes around the value.
- If you re-declare a JavaScript variable, it will not lose its value.
- = is used to assign values.
- + is used to add values.
- JavaScript has three kinds of popup boxes: Alert box, Confirm box, and Prompt box.
- An alert box is often used if you want to make sure information comes through to the user.
- A confirm box is often used if you want the user to verify or accept something.
- A prompt box is often used if you want the user to input a value before entering a page.
- A function will be executed by an event or by a call to the function.
- Loops execute a block of code a specified number of times, or while a specified condition is true.
- Events are actions that can be detected by JavaScript.
- Events are normally used in combination with functions, and the function will not be executed before the event occurs
- The try...catch statement allows you to test a block of code for errors.
- The throw statement allows you to create an exception.
- In JavaScript you can add special characters to a text string by using the backslash sign.
- JavaScript ignores extra spaces.
- You can break up a code line within a text string with a backslash.

Chapter 38 – jQuery

jQuery is a **cross-platform, cross-browser JavaScript library** designed to simplify the client-side scripting of HTML. This page lists down all the jQuery APIs at one place for your easy access.

jQuery – Selectors

Selector	Description
Name	Selects all elements which match with the given **elementName**.
#ID	Selects a single element which matches with the given **ID**
.Class	Selects all elements which match with the given **Class**.
Universal (*)	Selects all elements available in a DOM.
Multiple Elements B, C	Selects the combined results of all the specified selectors **B or C**.

Similar to above syntax and examples, following examples would give you understanding on using different type of other useful selectors:

- **$('*'):** This selector selects all elements in the document.
- **$("p > *"):** This selector selects all elements that are children of a paragraph element.
- **$("#specialID"):** This selector function gets the element with id="specialID".
- **$(".specialClass"):** This selector gets all the elements that have the class of*specialClass*.
- **$("li:not(.myclass)"):** Selects all elements matched by <li> that do not have class="myclass".
- **$("a#specialID.specialClass"):** This selector matches links with an id of *specialID* and a class of *specialClass*.
- **$("p a.specialClass"):** This selector matches links with a class of *specialClass* declared within <p> elements.
- **$("ul li:first"):** This selector gets only the first <li> element of the <ul>.
- **$("#container p"):** Selects all elements matched by <p> that are descendants of an element that has an id of *container*.
- **$("li > ul"):** Selects all elements matched by <ul> that are children of an element matched by
- **$("strong + em"):** Selects all elements matched by that immediately follow a sibling element matched by .
- **$("p ~ ul"):** Selects all elements matched by <ul> that follow a sibling element matched by <p>.

- **$("code, em, strong"):** Selects all elements matched by <code> or or .
- **$("p strong, .myclass"):** Selects all elements matched by that are descendants of an element matched by <p> as well as all elements that have a class of*myclass*.
- **$(":empty"):** Selects all elements that have no children.
- **$("p:empty"):** Selects all elements matched by <p> that have no children.
- **$("div[p]"):** Selects all elements matched by that contain an element matched by <p>.
- **$("p[.myclass]"):** Selects all elements matched by <p> that contain an element with a class of *myclass*.
- **$("a[@rel]"):** Selects all elements matched by <a> that have a rel attribute.
- **$("input[@name=myname]"):** Selects all elements matched by <input> that have a name value exactly equal to *myname.*
- **$("input[@name^=myname]"):** Selects all elements matched by <input> that have a name value beginning with *myname*.
- **$("a[@rel$=self]"):** Selects all elements matched by <p> that have a class value ending with *bar*
- **$("a[@href*=domain.com]"):** Selects all elements matched by <a> that have an href value containing domain.com.
- **$("li:even"):** Selects all elements matched by <li> that have an even index value.
- **$("tr:odd"):** Selects all elements matched by <tr> that have an odd index value.
- **$("li:first"):** Selects the first <li> element.
- **$("li:last"):** Selects the last <li> element.
- **$("li:visible"):** Selects all elements matched by <li> that are visible.
- **$("li:hidden"):** Selects all elements matched by <li> that are hidden.
- **$(":radio"):** Selects all radio buttons in the form.
- **$(":checked"):** Selects all checked boxex in the form.
- **$(":input"):** Selects only form elements (input, select, textarea, button).
- **$(":text"):** Selects only text elements (input[type=text]).
- **$("li:eq(2)"):** Selects the third <li> element
- **$("li:eq(4)"):** Selects the fifth <li> element
- **$("li:lt(2)"):** Selects all elements matched by <li> element before the third one; in other words, the first two <li> elements.
- **$("p:lt(3)"):** selects all elements matched by <p> elements before the fourth one; in other words the first three <p> elements.
- **$("li:gt(1)"):** Selects all elements matched by <li> after the second one.
- **$("p:gt(2)"):** Selects all elements matched by <p> after the third one.
- **$("div/p"):** Selects all elements matched by <p> that are children of an element matched by .
- **$("div//code"):** Selects all elements matched by <code>that are descendants of an element matched by .
- **$("//p//a"):** Selects all elements matched by <a> that are descendants of an element matched by <p>
- **$("li:first-child"):** Selects all elements matched by <li> that are the first child of their parent.
- **$("li:last-child"):** Selects all elements matched by <li> that are the last child of their parent.
- **$(":parent"):** Selects all elements that are the parent of another element, including text.
- **$("li:contains(second)"):** Selects all elements matched by <li> that contain the text second.

jQuery - DOM Attributes

Methods	Description
attr(properties)	Set a key/value object as properties to all matched elements.
attr(key, fn)	Set a single property to a computed value, on all matched elements.
removeAttr(name)	Remove an attribute from each of the matched elements.
hasClass(class)	Returns true if the specified class is present on at least one of the set of matched elements.
removeClass(class)	Removes all or the specified class(es) from the set of matched elements.
toggleClass(class)	Adds the specified class if it is not present, removes the specified class if it is present.
html( )	Get the html contents (innerHTML) of the first matched element.
html(val)	Set the html contents of every matched element.
text( )	Get the combined text contents of all matched elements.
text(val)	Set the text contents of all matched elements.
val( )	Get the input value of the first matched element.
val(val)	Set the value attribute of every matched element if it is called on <input> but if it is called on <select> with the passed <option> value then passed option would be selected, if it is called on check box or radio box then all the matching check box and radiobox would be checked.

Similar to above syntax and examples, following examples would give you understanding on using various attribute methods in different situation:

- **$("#myID").attr("custom") :** This would return value of attribute *custom* for the first element matching with ID myID.
- **$("img").attr("alt", "Sample Image"):** This sets the **alt** attribute of all the images to a new value "Sample Image".
- **$("input").attr({ value: "", title: "Please enter a value" }); :** Sets the value of all <input> elements to the empty string, as well as sets the title to the string *Please enter a value*.
- **$("a[href^=http://]").attr("target","_blank"):** Selects all links with an href attribute starting with *http://* and set its target attribute to *_blank*
- **$("a").removeAttr("target") :** This would remove *target* attribute of all the links.
- **$("form").submit(function() {$(":submit",this).attr("disabled", "disabled");}); :** This would modify the disabled attribute to the value "disabled" while clicking Submit button.
- **$("p:last").hasClass("selected"):** This return true if last <p> tag has associated class*selected*.
- **$("p").text():** Returns string that contains the combined text contents of all matched <p> elements.
- **$("p").text("Hello World"):** This would set "<I>Hello World</I>" as text content of the matching <p> elements
- **$("p").html() :** This returns the HTML content of the all matching paragraphs.

- **$("div").html("Hello World") :** This would set the HTML content of all matching to *Hello World*.
- **$("input:checkbox:checked").val() :** Get the first value from a checked checkbox
- **$("input:radio[name=bar]:checked").val():** Get the first value from a set of radio buttons
- **$("button").val("Hello") :** Sets the value attribute of every matched element <button>.
- **$("input").val("on") :** This would check all the radio or check box button whose value is "on".
- **$("select").val("Orange") :** This would select Orange option in a dropdown box with options Orange, Mango and Banana.
- **$("select").val("Orange", "Mango") :** This would select Orange and Mango options in a dropdown box with options Orange, Mango and Banana.

jQuery - DOM Traversing

Selector	Description
eq(index)	Reduce the set of matched elements to a single element.
filter(selector)	Removes all elements from the set of matched elements that do not match the specified selector(s).
filter(fn)	Removes all elements from the set of matched elements that do not match the specified function.
is(selector)	Checks the current selection against an expression and returns true, if at least one element of the selection fits the given selector.
map(callback)	Translate a set of elements in the jQuery object into another set of values in a jQuery array (which may, or may not contain elements).
not(selector)	Removes elements matching the specified selector from the set of matched elements.
slice(start, [end])	Selects a subset of the matched elements.
add(selector)	Adds more elements, matched by the given selector, to the set of matched elements.
andSelf( )	Add the previous selection to the current selection.
children([selector])	Get a set of elements containing all of the unique immediate children of each of the matched set of elements.
closest(selector)	Get a set of elements containing the closest parent element that matches the specified selector, the starting element included.
contents( )	Find all the child nodes inside the matched elements (including text nodes), or the content document, if the element is an iframe.
end( )	Revert the most recent 'destructive' operation, changing the set of matched elements to its previous state .
find(selector)	Searches for descendent elements that match the specified selectors.
next([selector])	Get a set of elements containing the unique next siblings of each of the given set of elements.
nextAll([selector])	Find all sibling elements after the current element.
offsetParent( )	Returns a jQuery collection with the positioned parent of the first

	matched element.
parent([selector])	Get the direct parent of an element. If called on a set of elements, parent returns a set of their unique direct parent elements.
parents([selector])	Get a set of elements containing the unique ancestors of the matched set of elements (except for the root element).
prev([selector])	Get a set of elements containing the unique previous siblings of each of the matched set of elements.
prevAll([selector])	Find all sibling elements in front of the current element.
siblings([selector])	Get a set of elements containing all of the unique siblings of each of the matched set of elements.

jQuery - CSS Methods

Method	Description
css(name)	Return a style property on the first matched element.
css(name, value)	Set a single style property to a value on all matched elements.
css(properties)	Set a key/value object as style properties to all matched elements.
height(val)	Set the CSS height of every matched element.
height( )	Get the current computed, pixel, height of the first matched element.
innerHeight( )	Gets the inner height (excludes the border and includes the padding) for the first matched element.
innerWidth( )	Gets the inner width (excludes the border and includes the padding) for the first matched element.
offset( )	Get the current offset of the first matched element, in pixels, relative to the document
offsetParent( )	Returns a jQuery collection with the positioned parent of the first matched element.
outerHeight([margin])	Gets the outer height (includes the border and padding by default) for the first matched element.
outerWidth([margin])	Get the outer width (includes the border and padding by default) for the first matched element.
position( )	Gets the top and left position of an element relative to its offset parent.
scrollLeft(val)	When a value is passed in, the scroll left offset is set to that value on all matched elements.
scrollLeft( )	Gets the scroll left offset of the first matched element.
scrollTop(val)	When a value is passed in, the scroll top offset is set to that value on all matched elements.

scrollTop( )	Gets the scroll top offset of the first matched element.
width(val)	Set the CSS width of every matched element.
width( )	Get the current computed, pixel, width of the first matched element.

jQuery - DOM Manipulation Methods

Method	Description
after(content)	Insert content after each of the matched elements.
append(content)	Append content to the inside of every matched element.
appendTo(selector)	Append all of the matched elements to another, specified, set of elements.
before(content)	Insert content before each of the matched elements.
clone(bool)	Clone matched DOM Elements, and all their event handlers, and select the clones.
clone( )	Clone matched DOM Elements and select the clones.
empty( )	Remove all child nodes from the set of matched elements.
html(val)	Set the html contents of every matched element.
html( )	Get the html contents (innerHTML) of the first matched element.
insertAfter(selector)	Insert all of the matched elements after another, specified, set of elements.
insertBefore(selector)	Insert all of the matched elements before another, specified, set of elements.
prepend(content)	Prepend content to the inside of every matched element.
prependTo(selector)	Prepend all of the matched elements to another, specified, set of elements.
remove(expr)	Removes all matched elements from the DOM.
replaceAll(selector)	Replaces the elements matched by the specified selector with the matched elements.
replaceWith(content)	Replaces all matched elements with the specified HTML or DOM elements.
text(val)	Set the text contents of all matched elements.
text( )	Get the combined text contents of all matched elements.
wrap(elem)	Wrap each matched element with the specified element.
wrap(html)	Wrap each matched element with the specified HTML content.
wrapAll(elem)	Wrap all the elements in the matched set into a single wrapper element.

wrapAll(html)	Wrap all the elements in the matched set into a single wrapper element.
wrapInner(elem)	Wrap the inner child contents of each matched element (including text nodes) with a DOM element.
wrapInner(html)	Wrap the inner child contents of each matched element (including text nodes) with an HTML structure.

jQuery - Events Handling

Event Type	Description
blur	Occurs when the element loses focus
change	Occurs when the element changes
click	Occurs when a mouse click
dblclick	Occurs when a mouse double-click
error	Occurs when there is an error in loading or unloading etc.
focus	Occurs when the element gets focus
keydown	Occurs when key is pressed
keypress	Occurs when key is pressed and released
keyup	Occurs when key is released
load	Occurs when document is loaded
mousedown	Occurs when mouse button is pressed
mouseenter	Occurs when mouse enters in an element region
mouseleave	Occurs when mouse leaves an element region
mousemove	Occurs when mouse pointer moves
mouseout	Occurs when mouse pointer moves out of an element
mouseover	Occurs when mouse pointer moves over an element
mouseup	Occurs when mouse button is released
resize	Occurs when window is resized
scroll	Occurs when window is scrolled
select	Occurs when a text is selected
submit	Occurs when form is submitted
unload	Occurs when documents is unloaded

jQuery – AJAX

Methods and Description
jQuery.ajax(options) Load a remote page using an HTTP request.
jQuery.ajaxSetup(options) Setup global settings for AJAX requests.
jQuery.get(url, [data], [callback], [type]) Load a remote page using an HTTP GET request.
jQuery.getJSON(url, [data], [callback]) Load JSON data using an HTTP GET request.
jQuery.getScript(url, [callback]) Loads and executes a JavaScript file using an HTTP GET request.
jQuery.post(url, [data], [callback], [type]) Load a remote page using an HTTP POST request.
load(url, [data], [callback]) Load HTML from a remote file and inject it into the DOM.
serialize( ) Serializes a set of input elements into a string of data.
serializeArray( ) Serializes all forms and form elements like the .serialize() method but returns a JSON data structure for you to work with.

Based on different events/stages following methods are available:

Methods and Description
ajaxComplete(callback) Attach a function to be executed whenever an AJAX request completes.
ajaxStart(callback) Attach a function to be executed whenever an AJAX request begins and there is none already active.
ajaxError(callback) Attach a function to be executed whenever an AJAX request fails.
ajaxSend(callback) Attach a function to be executed before an AJAX request is sent.
ajaxStop(callback) Attach a function to be executed whenever all AJAX requests have ended.
ajaxSuccess(callback) Attach a function to be executed whenever an AJAX request completes successfully.

jQuery – Effects

Methods and Description
animate(params, [duration, easing, callback]) A function for making custom animations.
animate(params, options) A function for making custom animations.
fadeIn(speed, [callback]) Fade in all matched elements by adjusting their opacity and firing an optional callback after completion.
fadeOut(speed, [callback]) Fade out all matched elements by adjusting their opacity to 0, then setting display to "none" and firing an optional callback after completion.
fadeToggle(speed, [callback]) This function toggles between fadeIn and fadeOut
fadeTo(speed, opacity, callback) Fade the opacity of all matched elements to a specified opacity and firing an optional callback after completion.
hide( ) Hides each of the set of matched elements if they are shown.
hide(speed, [callback]) Hide all matched elements using a graceful animation and firing an optional callback after completion.
show( ) Displays each of the set of matched elements if they are hidden.
show(speed, [callback]) Show all matched elements using a graceful animation and firing an optional callback after completion.
slideDown(speed, [callback]) Reveal all matched elements by adjusting their height and firing an optional callback after completion.
slideToggle(speed, [callback]) Toggle the visibility of all matched elements by adjusting their height and firing an optional callback after completion.
slideUp(speed, [callback]) Hide all matched elements by adjusting their height and firing an optional callback after completion.
stop([clearQueue, gotoEnd]) Stops all the currently running animations on all the specified elements.
toggle( ) Toggle displaying each of the set of matched elements.
toggle(speed, [callback]) Toggle displaying each of the set of matched elements using a graceful animation and firing an optional callback after completion.

toggle(switch) Toggle displaying each of the set of matched elements based upon the switch (true shows all elements, false hides all elements).
jQuery.fx.off Globally disable all animations.

UI Library Based Effects:

To use these effects you would have to download jQuery UI Library **jquery-ui-1.7.2.custom.min.js** or latest version of this UI library from jQuery UI Library.

Methods and Description
Blind Blinds the element away or shows it by blinding it in.
Bounce Bounces the element vertically or horizontally n-times.
Clip Clips the element on or off, vertically or horizontally.
Drop Drops the element away or shows it by dropping it in.
Explode Explodes the element into multiple pieces.
Fold Folds the element like a piece of paper.
Highlight Highlights the background with a defined color.
Puff Scale and fade out animations create the puff effect.
Pulsate Pulsates the opacity of the element multiple times.
Scale Shrink or grow an element by a percentage factor.
Shake Shakes the element vertically or horizontally n-times.
Size Resize an element to a specified width and height.
Slide Slides the element out of the viewport.
Transfer Transfers the outline of an element to another.

Chapter 39 – AngularJS

Conceptual Overview

AngularJS is not a library. It's a JavaScript framework that embraces extending HTML into a more expressive and readable format.

Concept	Description
Template	HTML with additional markup
Directives	extend HTML with custom attributes and elements
Model	the data shown to the user in the view and with which the user interacts
Scope	context where the model is stored so that controllers, directives and expressions can access it
Expressions	access variables and functions from the scope
Compiler	parses the template and instantiates directives and expressions
Filter	formats the value of an expression for display to the user
View	what the user sees (the DOM)
Data Binding	sync data between the model and the view
Controller	the business logic behind views
Dependency Injection	Creates and wires objects and functions
Injector	dependency injection container
Module	a container for the different parts of an app including controllers, services, filters, directives which configures the Injector
Service	reusable business logic independent of views

Chapter 40 – AJAX, JSON and DOJO

AJAX (Asynchronous JavaScript and XML)

AJAX allows web pages to be updated asynchronously by exchanging small amounts of the data with the server behind the scenes, without reloading the whole page.

- AJAX is a technique for creating fast and dynamic web pages
- AJAX applications are browser and platform-independent
- With AJAX, web applications can retrieve data from the server asynchronously in the background without interfering with the display and behavior of the existing page
- Data is usually retrieved using the XMLHttpRequest object
- Despite the name, the use of XML is not needed, and the requests need not be asynchronous

AJAX is based on Internet standards, and uses a combination of:

-
- XMLHttpRequest object(to exchange the data asynchronously with a server)
- JavaScript/DOM (to display/interact with the data)
- CSS (to style the data)
- XML (often used as the format for transferring data)

Create an XMLHttpRequest Object:

```
If(window.XMLHttpRequest) {
   XMLHttp = new XMLHttpRequest();
}
else // IE 5 or 6
XMLHttp = new ActiveXObject("Microsoft.XMLHTTP");
}
```

Send a Request to a Server:

```
open(method,URL, async)
send(string for POST)
```

method - GET or POST

GET:

1. Simpler and Faster than POST
2. Used frequently

POST:
1. A cached file is not an option(update file or database)
2. Sending a large amount of data(has no limitations)
3. Sending user input(unknown characters)
4. Robust and Secure than GET

```
XMLHttp.open("GET", "demo.JSP", true);
XMLHttp.send();

XMLHttp.open("POST", "demo.JSP", true);
XMLHttp.setRequestHeader("Content-type","application/x-www-form-urlencoded");
XMLHttp.send(prop=val&prop=Val);
```

ASYNC=true

```
XMLHttp.onreadystatechange=function (){
if(XMLHttp.readyState == 4 && XMLHttp.status == 200) {
 document.getElementById("div").innerHTML=XMLHttp.responseText;
}
}
XMLHttp.open("GET", "demo.JSP", true);
XMLHttp.send();
```

ASYNC=false

```
XMLHttp.open("GET", "demo.JSP", true);
XMLHttp.send();
document.getElementById("div").innerHTML=XMLHttp.responseText;
```

Server Response:

1. responseText
2. responseXML

Ready State:

onreadystatechange event
- Stores a function(or the name of a function) to be called automatically each time the readyState property changes

readyState property
- Holds the status of the XMLHttpRequest. Changes from 0 to 4.
0: request not initialized
1: server connection established
2: request received
3: processing request
4: request finished and response is ready

status property
- 200: "OK"
  404: Page not Found

Using a Callback Function

A callback function is a function passed as a parameter to another function.

Using responseXML property:

```
XmlDoc = XMLHttp.responseXML.documentElement;
Tags = XmlDoc.getElementByTagName("name");

For(int I=0; I<tags.length; I++){
 txt= Tags[I].firstChild.nodeValue; or. txt=Tags[I].childNodes[0].nodeValue;
}
document.getElementById("div").innerHTML=txt;
```

JSON (JavaScript Object Notation)

JSON is a Java library that helps convert Java objects into a string representation. This string, when eval()ed in JavaScript, produces an array that contains all of the information that the Java object contained. JSON's object notation grammar is suitable for encoding many nested object structures. Since this grammar is much smaller than its XML counterpart, and given the convenience of the eval() function, it is an ideal choice for fast and efficient data transport between browser and server.

JSON is designed to be used in conjunction with JavaScript code making HTTP requests. Since server-side code can be written in a variety of languages, JSON is available in many different languages such as C#, Python, PHP, and of course, Java.

DOJO

Dojo is a set of powerful JavaScript libraries that provide a simple API to a plethora of features. One of these features is the ability to make HTTP requests and receive their responses. This is the main functionality of Dojo that we will utilize. Aside from providing Ajax functionality, Dojo also provides packages for string manipulation, DOM manipulation, drag-and-drop support, and data structures such as lists, queues, and stacks.

Chapter 41 – UNIX Shell Script

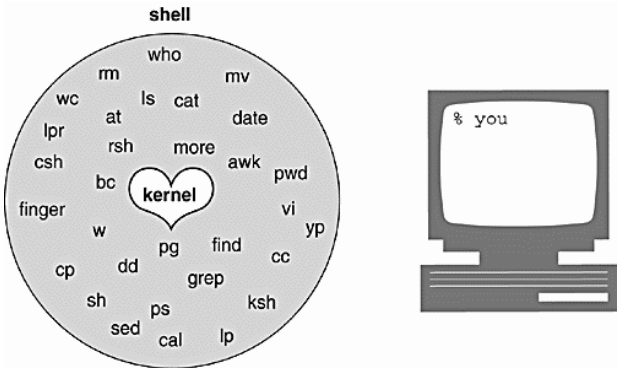
UNIX

UNIX stands for UNiplexed Information and Computing System. It was originally spelled "Unics."
It is a powerful, multi-user environment that has been implemented on a variety of platforms. Once the domain of servers and advanced users, it has become accessible to novices as well through the popularity of Linux, Solaris and Mac OS X. With the notable exception of Microsoft Windows, all current major operating systems have some kind of UNIX at their cores.

Shell

The shell is a special program used as an interface between the user and the heart of the UNIX operating system, a program called the kernel, as shown in Figure 1.1. The kernel is loaded into memory at boot-up time and manages the system until shutdown. It creates and controls processes, and manages memory, file systems, communications, and so forth. All other programs, including shell programs, reside out on the disk. The kernel loads those programs into memory, executes them, and cleans up the system when they terminate. The shell is a utility program that starts up when you log on. It allows users to interact with the kernel by interpreting commands that are typed either at the command line or in a script file.

Figure 1.1. The kernel, the shell, and you.



The Three Major UNIX Shells

The three prominent and supported shells on most UNIX systems are

1. the Bourne shell (AT&T shell),
2. the C shell (Berkeley shell), and
3. the Korn shell (superset of the Bourne shell).

The Bourne shell is the standard UNIX shell, and is used to administer the system. Most of the system administration scripts, such as the rc start and stop scripts and shutdown are Bourne shell scripts, and when in single user mode, this is the shell commonly used by the administrator when running as root. This shell was written at AT&T and is known for being concise, compact, and fast. The default Bourne shell prompt is the dollar sign ($).

The C shell was developed at Berkeley and added a number of features, such as command line history, aliasing, built-in arithmetic, filename completion, and job control. The C shell has been favored over the Bourne shell by users running the shell interactively, but administrators prefer the Bourne shell for scripting, because Bourne shell scripts are simpler and faster than the same scripts written in C shell. The default C shell prompt is the percent sign (%).

The Korn shell is a superset of the Bourne shell written by David Korn at AT&T. A number of features were added to this shell above and beyond the enhancements of the C shell. Korn shell features include an editable history, aliases, functions, regular expression wildcards, built-in arithmetic, job control, co-processing, and special debugging features. The Bourne shell is almost completely upward-compatible with the Korn shell, so older Bourne shell programs will run fine in this shell. The default Korn shell prompt is the dollar sign ($).

Responsibilities of the Shell

The shell is ultimately responsible for making sure that any commands typed at the prompt get properly executed. Included in those responsibilities are:

1. Reading input and parsing the command line.
2. Evaluating special characters.
3. Setting up pipes (|), redirection (>), and background processing (&).
4. Handling signals.
5. Setting up programs for execution.

Types of Commands

When the shell is ready to execute the command, it evaluates command types in the following order:

1. Aliases
2. Keywords
3. Functions (bash)
4. Built-in commands
5. Executable programs

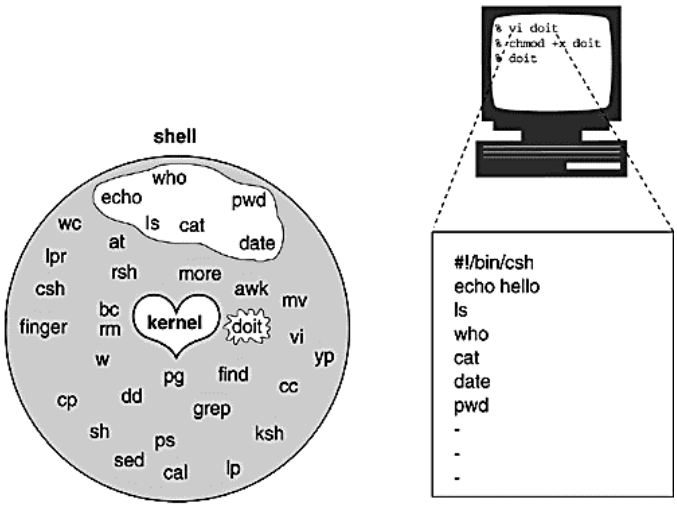
Parsing the Command Line

1. History substitution is performed (if applicable).
2. Command line is broken up into tokens, or words.
3. History is updated (if applicable).
4. Quotes are processed.
5. Alias substitution and functions are defined (if applicable).
6. Redirection, background, and pipes are set up.
7. Variable substitution ($user, $name, etc.) is performed.
8. Command substitution (echo for today is 'date') is performed.
9. Filename substitution, called globbing (cat abc.??, rm *.c, etc.) is performed.
10. Program execution.

Shell Script

A shell script is a script written for the shell, or command line interpreter, of an operating system. The shell is often considered a simple domain-specific programming language. The typical operations performed by shell scripts include file manipulation, program execution, and printing text.

Creating a generic shell script



I. Introduction to Shell Scripts

A. The shell is the program that you run when you log in
B. It is a command interpreter
C. There are three standard shells - C, Korn and Bourne
D. Shell prompts users, accepts command, parses, then interprets command
E. Most common form of input is command line input
```
cat file1 file2 file3
```
F. Most commands are of the format
```
command [- option list] [argument list]
```
G. Redirection and such
1. < redirect input from standard input
2. > redirect output from standard output
3. >> redirect output and append
4. | "pipes" output from one command to another
```
ls -l | more
```
5. `tee` "pipes" output to file and standard out
```
ls -l | tee rpt2 | more
```
H. Entering commands
1. Multiple commands can be entered on the same line if separated by ;
2. Command can span multiple lines if \R is typed at the end of each line except the last (R stands for carriage return, i.e. ENTER). This is escape sequence.
I. Wild card characters can be used to specify file names in commands
1. * 0 or more characters
2. ? one character of any kind
3. [,] list of characters to match single character
J. Simplest scripts combine commands on single line like
```
ls -l | tee rpt2 | more
```
K. Slightly more complex script will combine commands in a file
1. Use any text editor to create file, say `my_sc`
2. Type commands into file, one per line (unless you use ; to seperate)
3. Save file
4. Make file readable and executable (more later on this)
```
chmod a+rx my_sc
```
5. run script by entering path to file
```
./my_sc
```
We will make this a little easier later
L. See examples 1 and 2

II. Variables

A. The statment name=value creates and assigns value to variable
```
SUM=12
```
B. Traditional to use all upper case characters for names
C. Access content of variable by preceding name with $
```
echo $SUM
```
D. Arguments go from right to left
E. Results of commands can be assigned to variables
```
SYS=`hostname`
```
F. Strings are anything delimited by ""
G. Variables used in strings are evaluated
H. See example 3
I. System/standard variables
1. Command line arguments
Accessed by `$1` through `$9` for the first 9 command line arguments. Can access more by using the `shift` command. This makes $1 .. $9 reference command line arguments 2-10. It can be repeated to access a long list of arguments.
2. `$#` number of arguments passed on the command line
3. `$-` Options currently in effect (supplied to sh or to set
4. `$*` all the command line arguments as one long double quoted string
5. `$@` all the command line arguments as a series of double quoted strings
6. `$?` exit status of previous command
7. `$$` PID ot this shell's process
8. `$!` PID of most recently started background job
9. `$0` First word, that is, name of command/script

III. Conditional Variable Substitution

A. `${var:-string}` Use var if set, otherwise use string
B. `${var:=string}` Use var if set, otherwise use string and assign string to var
C. `${var:?string}` Use var if set, otherwise print string and exit
D. `${var:+string}` Use string if var if set, otherwise use nothing

IV. Conditional

A. The *condition* part can be expressed two ways. Either as
```
test condition
```
or
```
[ condition ]
```
where the spaces are significant.
B. There are several conditions that can be tested for
1. `-s file` file greater than 0 length
2. `-r file` file is readable

3. `-w file` file is writable
4. `-x file` file is executable
5. `-f file` file exists and is a regular file
6. `-d file` file is a directory
7. `-c file` file is a character special file
8. `-b file` file is a block special file
9. `-p file` file is a named pipe
10. `-u file` file has SUID set
11. `-g file` file has SGID set
12. `-k file` file has sticky bit set
13. `-z string` length of string is 0
14. `-n string` length of string is greater than 0
15. `string1 = string2` string1 is equal to string2
16. `string1 != string2` string1 is different from string2
17. `string` string is not null
18. `int1 -eq int2` integer1 equals integer2
19. `int1 -ne int2` integer1 does not equal integer2
20. `int1 -gt int2` integer1 greater than integer2
21. `int1 -ge int2` integer1 greater than or equal to integer2
22. `int1 -lt int2` integer1 less than integer2
23. `int1 -le int2` integer1 less than or equal to integer2
24. `! condition` negates (inverts) condition
25. `cond1 -a cond2` true if condition1 and condition2 are both true
26. `cond1 -o cond2` true if either condition1 or condition2 are true
27. `\( \)` used to group complex conditions

V. Flow Control

A. The if statement

```
if condition
then
commands
else
commands
fi
```

B. Both the while and until type of loop structures are supported

```
while condition
do
commands
done

until condition
do
commands
done
```

C. The case statement is also supported

```
case string in
pattern1)
commands
;;

pattern2)
commands
;;

esac
```

The pattern can either be an integer or a single quoted string

The * is used as a catch-all default pattern

D. The for command

```
for var [in list]
do
commands
done
```

where either a list (group of double quoted strings) is specified, or $@ is used

VI. Other Commands

A. Output

1. Use the `echo` command to display data
2. `echo "This is some data"` will output the string
3. `echo "This is data for the file = $FILE"` will output the string and expand the variable first. The output from an `echo` command is automatically terminated with a newline.

B. Input

1. The `read` command reads a line from standard input
2. Input is parsed by whitespace, and assigned to each respective variable passed to the `read` command
3. If more input is present than variables, the last variable gets the remainder
4. If for instance the command was `read a b c` and you typed "Do you Grok it" in response, the variables would contain $a="Do", $b="you" $c="Grok it"

C. Set the value of variables $1 thru $n

1. If you do `set` `` `command` ``, then the results for the command will be assigned to each of the variables $1, $2, etc. parsed by whitespace

D. Evaluating expressions

1. The `expr` command is used to evaluate expressions

2. Useful for integer arithmetic in shell scripts `i=`expr $i +1``

E. Executing arguments as shell commands

1. The `eval` command executes its arguments as a shell command

VII. Shell functions

A. General format is

B. `function_name ()`

C. `{`

D. *commands*

E. `}`

VIII. Miscellaneous

A. \n at end of line continues on to next line

B. Metacharacters

1. * any number of characters
2. ? any one character
3. [.] list of alternate characters for one character position

C. Substitution

1. delimit with `` (back quote marks, generally top left corner of keyboard)
2. executes what is in `` and substitutes result in string

D. Escapes

1. \ single character
2. ' groups of characters
3. " groups of characters, but some special characters processed ($\`)

E. Shell options

1. Restricted shell sh -r

a. can't cd, modify PATH, specify full path names or redirect output

b. should not allow write permissions to directory

2. Changing shell options

a. Use set option +/- to turn option on/off

b. e interactive shell

c. f filename substitution

d. n run, no execution of commands

e. u unset variables as errors during substitution

f. x prints commands and arguments during execution

Examples

Single Line Script

```
#! bin/sh
# Script lists all files in current
directory in decending order by size

ls -l | sort -r -n  +4 -5
```

Multiline Script

```
#!/usr/bin/ksh
# Lists 10 largest files in current
directory by size

ls -l > /tmp/f1
sort -r -n +4 -5 /tmp/f1 > /tmp/f2
rm /tmp/f1
head /tmp/f2 > /tmp/f3
rm /tmp/f2
more /tmp/f3
rm /tmp/f3
```

```
#!/usr/bin/ksh
# Uses variables to store data from
commands

SYS=`hostname`
ME=`whoami`
W="on the system"
echo "I am $ME $W $SYS"
```

This is a quick reference guide to the meaning of some of the less easily guessed commands and codes.

Command	Description	Example
&	Run the previous command in the background	ls &
&&	Logical AND	if ["$foo" -ge "0"] && ["$foo" -le "9"]
||	Logical OR	if ["$foo" -lt "0"] \|\| ["$foo" -gt "9"] (not in Bourne shell)
^	Start of line	grep "^foo"
$	End of line	grep "foo$"
=	String equality (cf. -eq)	if ["$foo" = "bar"]
!	Logical NOT	if ["$foo" != "bar"]
$$	PID of current shell	echo "my PID = $$"
$!	PID of last background command	ls & echo "PID of ls = $!"
$?	exit status of last command	ls ; echo "ls returned code $?"
$0	Name of current command (as called)	echo "I am $0"
$1	Name of current command's first parameter	echo "My first argument is $1"
$9	Name of current command's ninth parameter	echo "My ninth argument is $9"
$@	All of current command's parameters (preserving whitespace and quoting)	echo "My arguments are $@"
$*	All of current command's parameters (not preserving whitespace and quoting)	echo "My arguments are $*"
-eq	Numeric Equality	if ["$foo" -eq "9"]
-ne	Numeric Inquality	if ["$foo" -ne "9"]
-lt	Less Than	if ["$foo" -lt "9"]
-le	Less Than or Equal	if ["$foo" -le "9"]
-gt	Greater Than	if ["$foo" -gt "9"]
-ge	Greater Than or Equal	if ["$foo" -ge "9"]
-z	String is zero length	if [-z "$foo"]
-n	String is not zero length	if [-n "$foo"]
-nt	Newer Than	if ["$file1" -nt "$file2"]
-d	Is a Directory	if [-d /bin]
-f	Is a File	if [-f /bin/ls]
-r	Is a readable file	if [-r /bin/ls]
-w	Is a writable file	if [-w /bin/ls]
-x	Is an executable file	if [-x /bin/ls]
parenthesis: (...)	Function definition	function myfunc() { echo hello }

If Necessity is the Mother of Invention then Curiosity is the Father and we're their Kids!